UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Curso: BIC01 - Introducción a la Computación

Docente: Mag. Americo Chulluncuy Reynoso

2026-I

Introducción al Sistema Informático

Sesión 1 - BIC01: Introducción a la Computación

Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación
Universidad Nacional de Ingeniería

Tabla de Contenidos

1 ¿Qué es la Programación?

2 Historia de la Computación por Generaciones

3 Sistemas Informáticos: Hardware y Software

4 Lenguajes de Programación

5 Introducción a C++

6 Resolución de Problemas de Programación

7 Herramientas de Programación

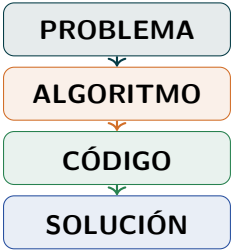
¿Qué es la Programación?

Definición

La **programación** consiste en escribir instrucciones precisas para que una computadora realice tareas específicas de forma automática.

Programar es como escribir una receta de cocina: debemos ser exactos, ordenados y anticipar cada paso que la máquina ejecutará.

- Resuelve problemas de forma sistemática
- Automatiza tareas repetitivas
- Crea herramientas y aplicaciones
- Es el lenguaje del siglo XXI



Historia de la Computación

- **1642** — Pascalina: primera calculadora mecánica (Blaise Pascal)
- **1833–35** — Máquina analítica de Charles Babbage
- **Ada Augusta Byron** — primera programadora de la historia
- **1942–46** — ENIAC: 18,000 tubos de vacío, 30 toneladas
- **1956** — Invención del transistor
- **1971** — Intel lanza el primer microprocesador
- **Hoy** — IA, Cloud y Quantum Computing

1ª Generación — Tubos de Vacío (1940–1956)

Características

- Componente base: **tubos de vacío** (válvulas electrónicas)
- Máquinas enormes: ocupaban habitaciones enteras
- Gran consumo energético y mucho calor
- Programación en **lenguaje de máquina** (binario)
- Sin sistema operativo

18,000 tubos
30 toneladas
170 m^2 de espacio
150 kW de potencia

Ejemplos

ENIAC (1945), **UNIVAC I** (1951), **IBM 701** (1952)

2ª Generación — Transistores (1956–1964)

Características

- Componente base: **transistor** (inventado en Bell Labs, 1947)
- Computadoras más pequeñas, rápidas y fiables
- Menor consumo de energía
- Surgen los **lenguajes ensamblador** y de alto nivel (FORTRAN, COBOL)
- Primeros sistemas operativos por lotes (*batch*)

Transistor

IBM 7090 (1959)

PDP-1 (1960)

Inicio de FORTRAN

Impacto clave

El transistor reemplazó los tubos de vacío: mismo trabajo, 1/100 del tamaño.

3ª Generación — Circuitos Integrados (1964–1971)

Características

- Componente base: **circuitos integrados** (chips)
- Cientos de transistores en una sola plaqueta de silicio
- Gran mejora en velocidad, tamaño y costo
- Surge el concepto de **familia de computadoras** (IBM System/360)
- Aparecen los **sistemas operativos modernos** y la multiprogramación
- Lenguajes: **C**, BASIC, Pascal

Circuito Integrado

IBM System/360

DEC PDP-8

Nace UNIX (1969)

4ª Generación — Microprocesadores (1971–2000)

Características

- Componente base: **microprocesador** (CPU en un solo chip)
- **Intel 4004** (1971): primer microprocesador comercial
- Nacimiento de las **computadoras personales (PC)**
- Apple II (1977), IBM PC (1981), Macintosh (1984)
- Sistemas operativos: MS-DOS, Windows, macOS, Linux
- Lenguajes: C++, Java, Python, SQL
- Nace la **World Wide Web** (Tim Berners-Lee, 1991)

Microprocesador

Intel 4004 → Pentium

PC masiva

Ley de Moore

5ª Generación — IA, Cloud y Quantum (2000–Presente)

Características generales

- Procesamiento paralelo masivo
- **Inteligencia Artificial** y aprendizaje automático
- **Cloud Computing**: infraestructura como servicio
- **Quantum Computing**: computación cuántica
- Dispositivos inteligentes e IoT
- Big Data y análisis en tiempo real

IA
ChatGPT, Gemini, Claude

Cloud
AWS, Azure, GCP

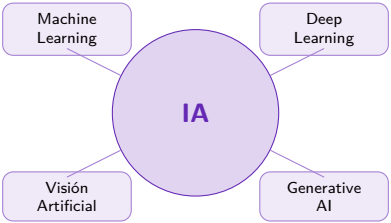
Quantum
IBM Q, Google Sycamore

5ª Generación — Inteligencia Artificial

¿Qué es la Inteligencia Artificial?

Capacidad de las máquinas para realizar tareas que normalmente requieren inteligencia humana: razonar, aprender, reconocer patrones y tomar decisiones.

- **Machine Learning**: modelos que aprenden de datos
- **Deep Learning**: redes neuronales profundas
- **NLP**: procesamiento de lenguaje natural
- **Visión computacional**: reconocimiento de imágenes
- **Generative AI**: ChatGPT, Claude, Gemini, DALL·E



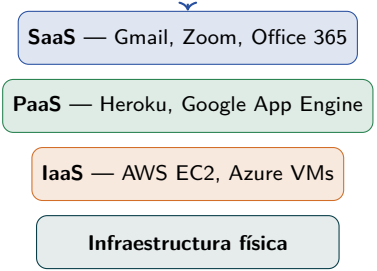
5ª Generación — Cloud Computing

¿Qué es Cloud Computing?

Entrega de servicios de cómputo (servidores, almacenamiento, bases de datos, redes, software) a través de Internet, pagando solo por lo que se usa.

Modelo	Descripción
IaaS	Infraestructura como servicio
PaaS	Plataforma como servicio
SaaS	Software como servicio

Proveedores líderes: **AWS**, **Microsoft Azure**, **Google Cloud**

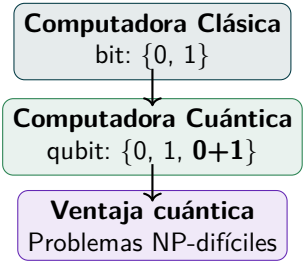


5ª Generación — Computación Cuántica

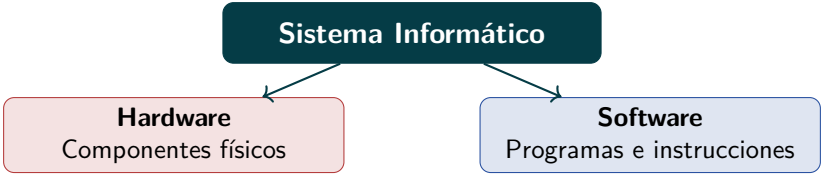
¿Qué es la Computación Cuántica?

Usa principios de la mecánica cuántica (**superposición** y **entrelazamiento**) para procesar información de formas imposibles para computadoras clásicas.

- **Bit clásico**: 0 ó 1
- **Qubit**: puede ser 0, 1 o ambos a la vez (superposición)
- Capaz de resolver en segundos problemas que llevarían millones de años en hardware clásico
- **Aplicaciones**: criptografía, simulación molecular, optimización, IA



¿Qué es un Sistema Informático?



Hardware

Todo lo que puedes **tocar**: CPU, RAM, disco duro, teclado, monitor, motherboard...

Software

Todo lo que **no puedes tocar**: sistema operativo, aplicaciones, compiladores, juegos...

Hardware — Componentes Principales

Dispositivos de entrada

- Teclado, mouse, micrófono
- Escáner, cámara web

Unidad Central de Procesamiento (CPU)

- **CU** — Unidad de Control
- **ALU** — Unidad Aritmético-Lógica
- **Registros** — memoria ultrarrápida interna

Memoria

- **RAM**: memoria de trabajo (volátil)
- **ROM**: firmware (no volátil)
- **Caché**: ultra-rápida, dentro del CPU

Almacenamiento secundario

- HDD (disco magnético)
- SSD (estado sólido, más rápido)
- USB, DVD, SD

Dispositivos de salida

- Monitor, impresora, parlantes

Placa principal (Motherboard)

Interconecta todos los componentes del sistema. Contiene el CPU, ranuras de RAM, buses y conectores.

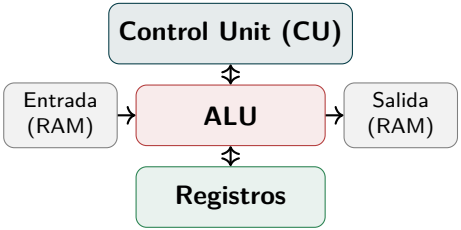
La CPU — El Cerebro del Computador

Unidad de Control (CU)

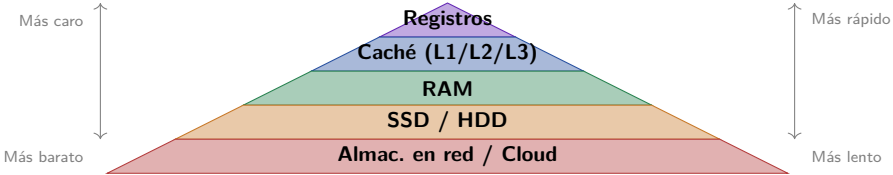
- Recupera instrucciones de la memoria (fetch)
- Decodifica las instrucciones (decode)
- Coordina todas las operaciones del sistema

Unidad Aritmético-Lógica (ALU)

- Operaciones aritméticas: $+$, $-$, $\times$, $/$
- Operaciones lógicas: AND, OR, NOT, XOR
- Comparaciones: $=$, $<$, $>$



Jerarquía de Memoria



Software — Clasificación

Software de Sistema

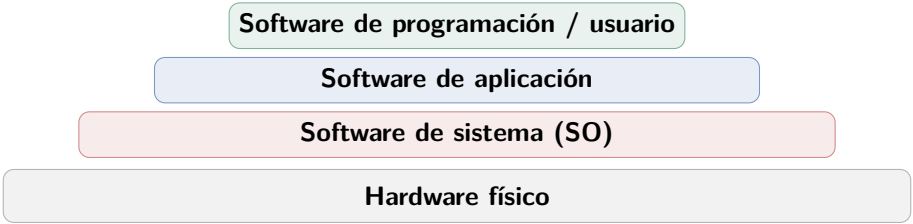
- Sistema operativo (SO)
- Controla el hardware
- Administra recursos
- **Ejemplos**: Windows, Linux, macOS, Android, iOS

Software de Aplicación

- Tareas del usuario final
- Corre sobre el SO
- **Ejemplos**: navegadores, Office, antivirus, reproductores, videojuegos

Software de Programación

- Herramientas para crear software
- **Ejemplos**: compiladores, IDEs, depuradores, PSeInt, VSCode, DevC++



Lenguajes de Programación — Definición

¿Qué es un lenguaje de programación?

Conjunto formal de **reglas sintácticas y semánticas** que permite a los programadores comunicarse con las computadoras para describir algoritmos y procesos.

Analogía lingüística:

- El lenguaje humano tiene gramática
- Los lenguajes de programación tienen **sintaxis**
- Las palabras tienen significado $\rightarrow$ las instrucciones tienen **semántica**

Lenguajes de programación?

Existen más de **700 lenguajes de programación** en el mundo. Sin embargo, en la práctica se usan menos de 20 de forma masiva.

Clasificación: Bajo Nivel vs Alto Nivel

Lenguajes de Bajo Nivel

- Próximos al hardware
- El programador controla la memoria directamente
- Muy eficientes pero difíciles de escribir y mantener
- **Lenguaje de máquina**: código binario (01001100...)
- **Ensamblador (ASM)**: mnemonics como `MOV AX, 1`

Lenguajes de Alto Nivel

- Cercanos al lenguaje humano
- Fáciles de aprender y mantener
- El compilador/intérprete los traduce a código de máquina
- **Ejemplos**: C++, Python, Java, JavaScript, Kotlin, Swift

Compilados vs Interpretados

Lenguajes Compilados

- El código fuente se traduce **completamente** antes de ejecutar
- Genera un archivo ejecutable
- Más rápidos en ejecución
- **Ejemplos**: C, C++, Rust, Go

Lenguajes Interpretados

- El código se traduce y ejecuta **línea por línea**
- Más flexibles y portables
- Más lentos (traducción en tiempo real)
- **Ejemplos**: Python, JavaScript, Ruby, PHP

¿Por qué C++?

Historia

- Creado por **Bjarne Stroustrup** en Bell Labs (1979–1985)
- Extensión del lenguaje **C** con orientación a objetos
- Primera versión oficial: **C++ 1985**
- Estándares: C++98 → C++11 → C++17 → C++20 → C++23

Áreas de uso

Videojuegos, sistemas operativos, compiladores, simulaciones científicas, motores gráficos, sistemas embebidos, software financiero de alta frecuencia.

Características clave

- ✔ Lenguaje de **propósito general**
- ✔ **Compilado**: alta performance
- ✔ **Multiparadigma**: estructurado + OOP + genérico
- ✔ Control directo de memoria (punteros)
- ✔ Biblioteca estándar rica (STL)
- ✔ Base para aprender otros lenguajes

Mi Primer Programa en C++

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hola, Mundo!" << endl;
    return 0;
}
```

Anatomía del programa

- `#include <iostream>` — incluye librería de entrada/salida
- `using namespace std` — acceso directo a funciones estándar
- `int main()` — función principal (punto de entrada)
- `cout «` — imprime en consola
- `endl` — salto de línea
- `return 0` — indica ejecución exitosa

C++ — Leer datos del usuario

```cpp
#include <iostream>
using namespace std;

int main() {
    int edad;
    cout << "Ingresa tu edad: ";
    cin >> edad;
    cout << "Tienes " << edad
         << " años." << endl;
    return 0;
}
```

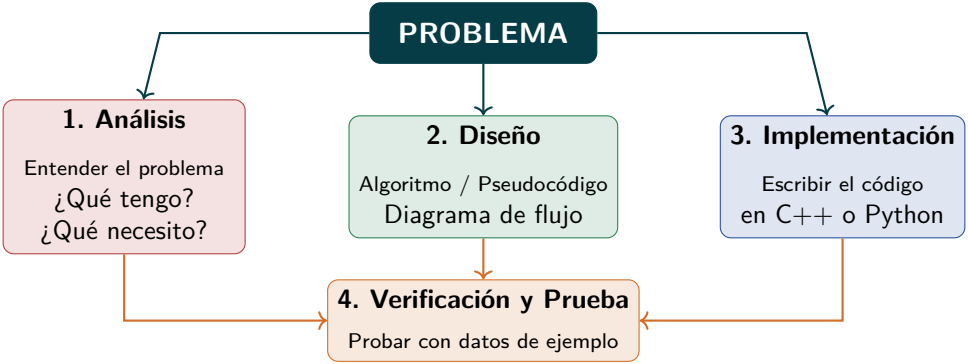
Conceptos nuevos

- `int edad` — declara variable entera
- `cin >>` — lee dato del teclado
- El flujo de datos: **teclado → variable → pantalla**

Tipos básicos de variables

`int`	enteros
`float`	decimales
`double`	decimales precisos
`char`	un carácter
`string`	texto
`bool`	verdadero/falso

¿Cómo Resolver un Problema de Programación?



Algoritmos — El Corazón de la Programación

¿Qué es un algoritmo?

Secuencia **finita**, **ordenada** y **no ambigua** de instrucciones para resolver un problema.

Propiedades de un buen algoritmo:

- **Finito**: termina en un número finito de pasos
- **Definido**: cada paso es claro y preciso
- **Entradas**: tiene datos de entrada (puede ser 0)
- **Salidas**: produce al menos un resultado
- **Efectivo**: cada paso es realizable

Ejemplo: calcular el promedio

Pseudocódigo (PSeInt):

```
Algoritmo Promedio
  Leer n1, n2, n3
  prom <- (n1+n2+n3)/3
  Escribir "Promedio:", prom
FinAlgoritmo
```

Herramientas que Usaremos en el Curso

PSeInt

- Pseudocódigo en español
- Ideal para aprender lógica antes de codificar
- Tiene diagramas de flujo automáticos
- Gratuito y multiplataforma

`pseint.sourceforge.io`

DevC++ (IDE)

- IDE completo para C/C++
- Editor + compilador integrado
- Fácil de instalar en Windows
- Bueno para comenzar

`Embarcadero DevC++`

VS Code (Recomendado)

- Editor moderno y extensible
- Soporte para C++, Python y más
- Extensiones útiles: IntelliSense, GitHub Copilot
- Multiplataforma (Win/Mac/Linux)

`code.visualstudio.com`

Flujo de trabajo en el curso

PSeInt (algoritmos) $\longrightarrow$ VS Code(código C++) $\longrightarrow$ Compilar $\longrightarrow$ Ejecutar

Próxima sesión

Algorimos, Variables y estructuras secuenciales en C++

achulluncuy@uni.edu.pe

Repositorio del curso

Aula virtual

"The Analytical Engine has no pretensions whatever to originate anything. It can do whatever we know how to order it to perform." — Ada Lovelace